

Programming and Databases

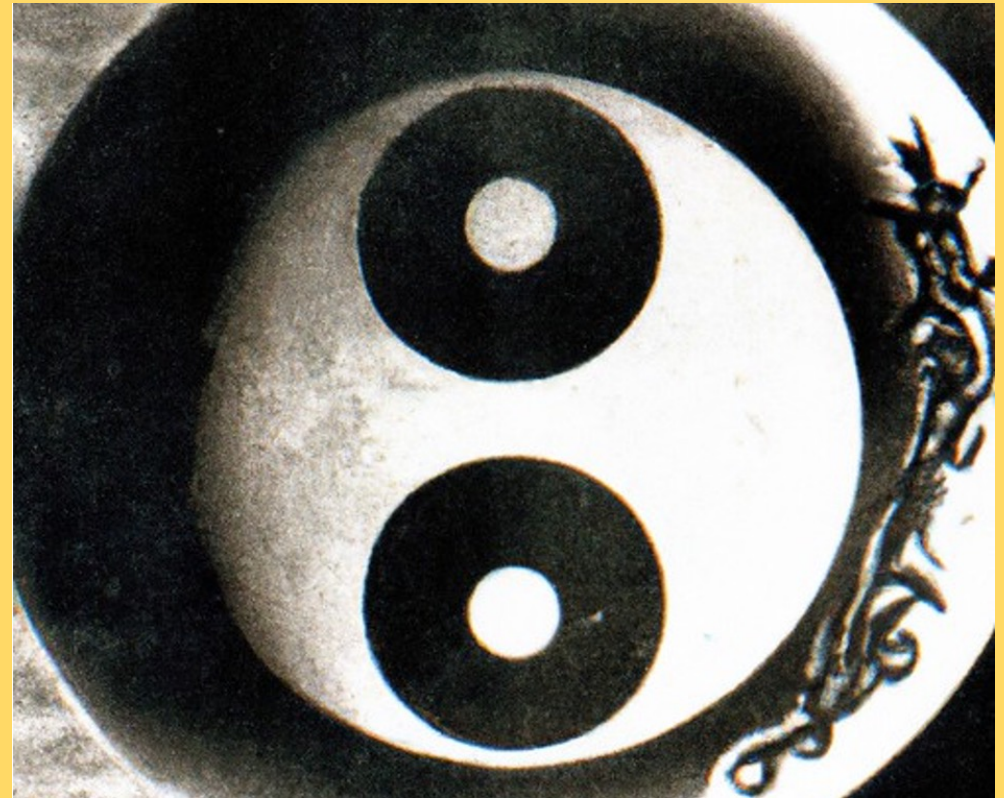
Joern Ploennigs

Functions

MIDJOURNEY: FUNCTIONAL LINES, REF. C.E.B. REAS

WIEDERHOLUNG: HÖRSAALFRAGE

What are if, elif, and else used for?



DALL·E 2: Yin and Yang in a ceramic dish

RECAP: IF, ELIF, ELSE

if, **elif**, **else** are commands **for** branching programs **in** Python

```
if Condition1:  
    print("Will run when Condition1 is true")  
elif Condition2:  
    print("Will run when Condition1 is false and Condition2 is true")  
else:  
    print("Will run when Condition1 is false and Condition2 is false")
```

RECAP: IN-CLASS QUESTION

What loop types exist?



Midjourney: Loop

RECAP: LOOPS

Loops are an important concept in programming for implementing repetitive or incremental solutions. In Python, the following are available:

- `for` (equivalent to a for-each loop)
- `while` (equivalent to a while loop)

Key properties:

- They work exactly like all other control statements—by indentation after a header line
- Unlike in functions, variables from loops are also available outside the loop!
- The other loop variants aren't explicitly present either, but can be emulated functionally

RECAP: WHILE LOOPS

While loop

In a While loop, the condition can be false right at the start. So the loop does not have to be executed.

```
Condition = True/False
while Condition:
    # Statement
    Condition = False
```

Do-While loop

In a Do-While loop, the condition is checked only at the end. The loop is therefore always executed at least once.

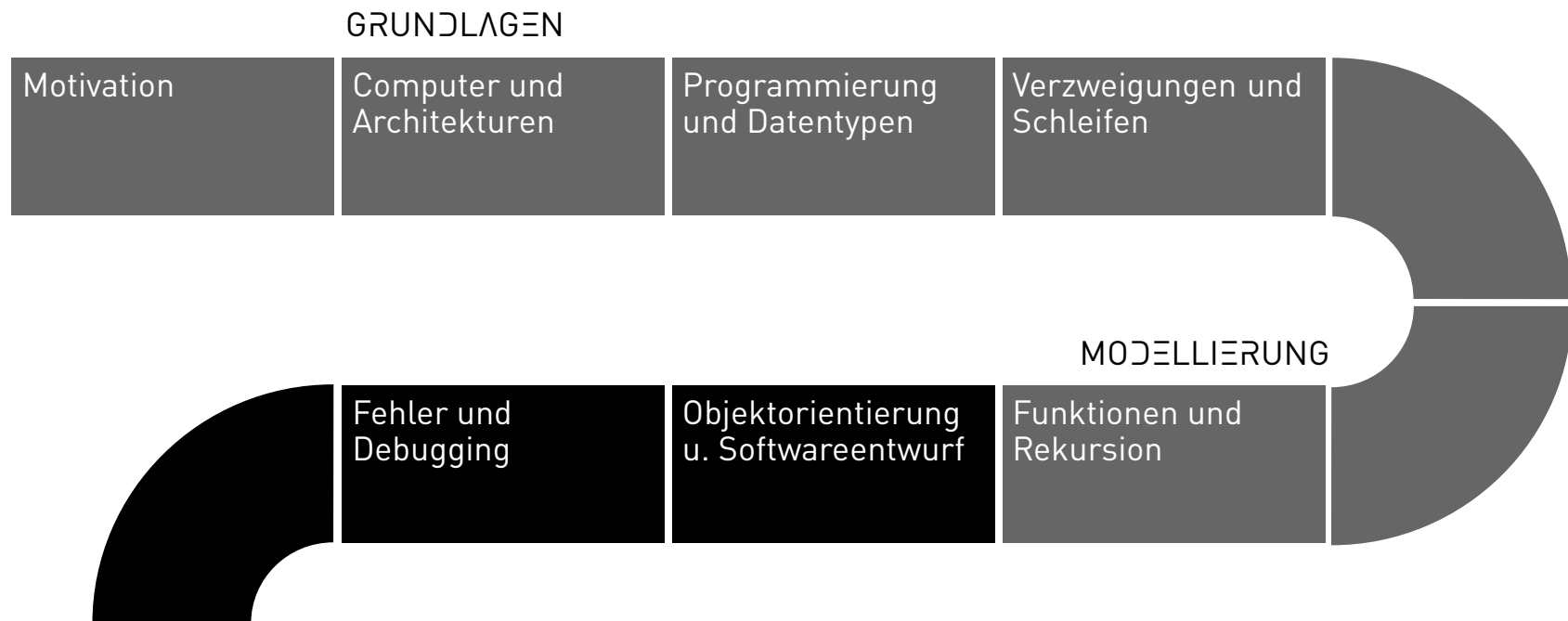
```
Condition = True
while Condition:
    # Statement
    Condition = False
```

Repeat-Until loop

In a Repeat-Until loop, the condition is also checked only at the end. The loop repeats until the condition becomes true.

```
Condition = False
while not Condition:
    # Statement
    Condition = True
```

PROCESS



FUNCTIONS – MATHEMATICAL

Functions in programming languages implement the mathematical concept of a function.

They represent a mapping from an input set to an output set.

Mapping: A function ***f*** assigns to every element ***x*** of a domain ***D*** an element ***y*** of a codomain ***Z***.

$$f : D \rightarrow Z, \quad x \mapsto y$$

FUNCTIONS – PROGRAMMING



Definition: Function

Reusable code that performs a specific task.

- Take a tuple of input values (*arguments*)
- Execute a fixed sequence of expressions and assignments
- Return a tuple of output values (*return values*)
- Are only executed when they are **called** in an expression

FUNCTIONS IN PYTHON

```
# Code block of the function definition  
def functionname(arg1, arg2): # <- the function name is selectable, arg1 and  
arg2 are arguments (parameters)  
    statement1 # <- The code block starts here and is indented  
    statement2 # <- Still part of the indented code block
```

- Function definitions begin with `def`
- It is followed by a name, arguments and a `:`
- Arguments are in scope for the entire function body
- When called, the arguments are bound to input data

FUNCTIONS WITH A RETURN VALUE

```
def functionName(arg1):  
    statement1  
    return outputValue
```

- `return` ends execution and returns the output value(s)
- `Returning` means: The return value replaces the function call in the original expression
- If no `return` is specified, the function returns `None`

EXAMPLE – DOUBLING A VALUE

Multiplication

```
x = arg1 * 2    # 4
x = arg1 * 4    # 16
x = arg1 * 8    # 128
```

Bit shifting (faster)

```
def times2(arg1):
    return arg1 << 1

x = arg1 << 1    # $2^1$
x = arg1 << 2    # $2^2$
x = arg1 << 3    # $2^3$
```

EXAMPLE – EUCLIDEAN DISTANCE

```
from math import sqrt

def distance(a1, a2, b1, b2):
    return sqrt((a1 - b1)**2 + (a2 - b2)**2)

a1, a2 = 2, 3
b1, b2 = 6, 6
x = distance(a1, a2, b1, b2) # 5.0
```

DEFAULT VALUES FOR PARAMETERS

```
def funktionsname(arg1, arg2="default"):  
    return statement1
```

- Arguments can have default values
- They are optional when calling; provided values override the default

EXAMPLE – APPENDING A UNIT TO A NUMBER

```
def measurement(number, unit="meters"):
    return str(number) + " " + unit

x = measurement(12)          # "12 meters"
x = measurement(5.5, "kg")  # "5.5 kg"
```

PASSING ARGUMENT VALUES (CONCEPTS)

Pass-by-value

- Only **values** are copied
- The passed-in variables are not accessible inside the function
- Safer, since there is no unexpected rebinding

Pass-by-reference

- Reference to the **same data**
- Values inside the function can be modified
- Saves time and memory

PASSING ARGUMENT VALUES IN PYTHON

- Hybrid variant, often called "**pass-by-assignment**"
- **Mutable** data types: behave like *pass-by-reference*
- **Immutable** data types: behave like *pass-by-value*
- If a *mutable* variable is rebound, it has no effect outside its scope

BUILT-IN FUNCTIONS

Python has an extensive list of built-in functions.

Function	Purpose
<code>print()</code> , <code>input()</code>	Output, input
<code>id()</code> , <code>type()</code>	Variable IDs, data types
<code>int()</code> , <code>str()</code> , <code>float()</code>	Data type conversion
<code>list()</code> , <code>tuple()</code> , <code>set()</code> , <code>dict()</code>	Complex data types
<code>len()</code>	Length of a complex data type
<code>abs()</code> , <code>max()</code> , <code>min()</code>	Mathematical basic functions
<code>exit()</code>	Exit the program
<code>sorted()</code>	Sorting
...	

WHAT ARE FUNCTIONS USED FOR?

- Readability, modularity, and reusability
"Write once, run anywhere"
- Modern style: break programs down into as many **core functions** as possible

Questions?

